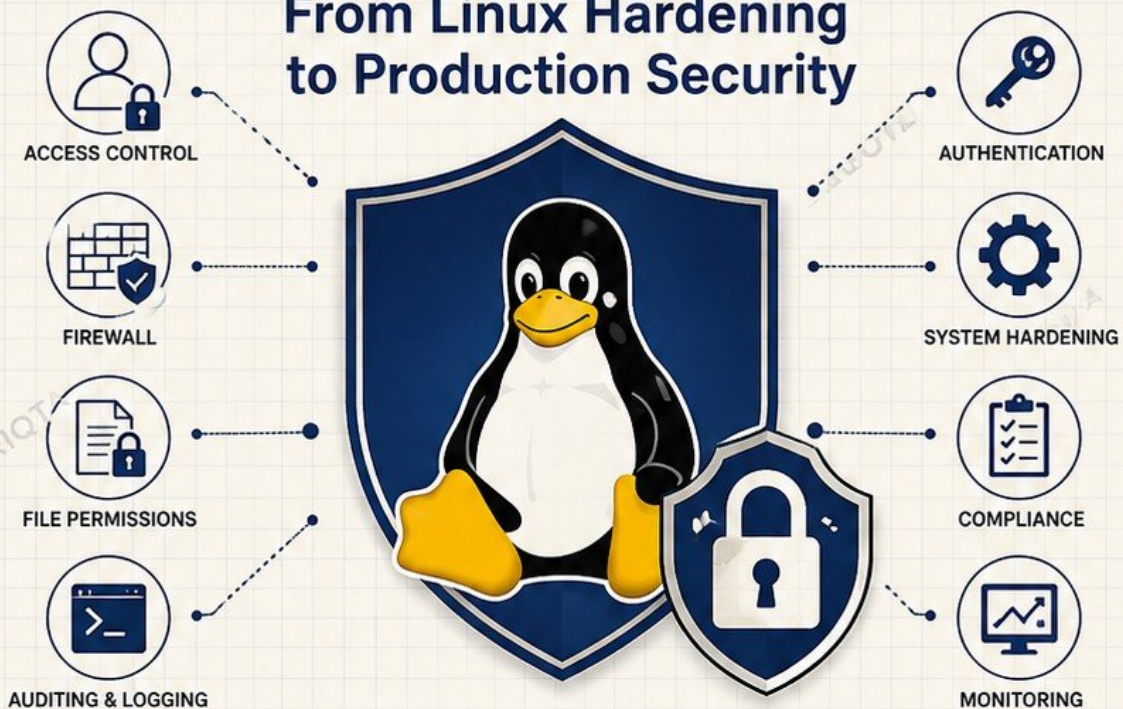


VERIQTA

Date: _____
Page No: _____

LINUX SECURITY HANDBOOK

From Linux Hardening
to Production Security



**SECURE TODAY
PROTECT TOMORROW**



HARDEN
Secure Systems



DETECT
Find Threats



PROTECT
Stop Attacks



RESPOND
Recover Fast

VERIQTA



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

VERIQTA

1. LINUX SECURITY FOUNDATIONS



STRONG LINUX SECURITY STARTS WITH UNDERSTANDING THE FUNDAMENTALS OF HOW SYSTEMS, USERS, PROCESSES, AND NETWORKS INTERACT.



LINUX SECURITY LAYERS

Linux security is built in multiple layers including the kernel, system services, filesystem, users, and network.



USERS, PROCESSES, FILES, SERVICES, AND NETWORK BOUNDARIES

Every component interacts. Securing each layer reduces risk across the entire system.



AUTHENTICATION VS AUTHORIZATION

Authentication verifies who you are.
Authorization determines what you are allowed to do.
Both must be configured correctly.



LEAST PRIVILEGE

Users, services, and processes should have only the minimum permissions required to perform their tasks.
Limit access. Reduce risk.



ATTACK SURFACE

Any point that can be used to attack your system.
Reduce exposed services, open ports, and unnecessary access to shrink the attack surface.



DEFENSE IN DEPTH

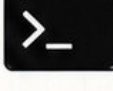
No single control is perfect. Use multiple layers of security so that if one layer fails, other layers continue to protect the system.



SECURITY BASELINE VS ONGOING MONITORING

A strong baseline sets the foundation.
Continuous monitoring detects changes, threats, and abnormal behavior early.

2. USERS, GROUPS, AND IDENTITY SECURITY

	LOCAL USERS AND SYSTEM ACCOUNTS Linux uses local users and system accounts to manage access and ownership. System accounts usually have low UIDs and no login shell.
	/ETC/PASSWD Stores user account information including username, UID, GID, home directory, and default shell.
	/ETC/SHADOW Stores encrypted passwords and password aging information. Access is restricted to the root user.
	/ETC/GROUP Stores group information including GID and member list.
	UID AND GID SECURITY Every user and group has a unique UID and GID. Protect UID 0 (root) and other privileged IDs.
	DETECTING UNEXPECTED ACCOUNTS Regularly check for unknown or suspicious user accounts and groups.
	LOCKING AND DISABLING ACCOUNTS Lock or disable unused or compromised accounts to prevent unauthorized access.
	REVIEWING PRIVILEGED USERS Review users with UID 0 and sudo access. Limit privileged access to only what is necessary.
	KEY MANAGEMENT COMMANDS Use Linux commands to inspect and manage users, groups, and account settings.

KEY ACCOUNT FILES

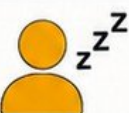
-  **/etc/passwd**
User account information
-  **/etc/shadow**
Encrypted passwords and aging info
-  **/etc/group**
Group information and members

 These files are critical for identity security. Protect them carefully.

KEY COMMANDS

-  **>_**
- id**
Show user and group identity information
- getent**
Query user and group database
- passwd**
Manage user passwords
- usermod**
Modify user account properties
- chage**
Manage password aging and expiry

3. PASSWORD AND ACCOUNT SECURITY

	PASSWORD AGING Defines the minimum number of days before a password can be changed again. Prevents frequent and weak password changes.
	PASSWORD EXPIRATION Forces users to change passwords after a specified number of days. Reduces risk from compromised passwords.
	ACCOUNT EXPIRATION Sets an expiration date for user accounts. Prevents unused or unnecessary accounts from remaining active.
	PAM PASSWORD POLICIES Pluggable Authentication Modules (PAM) enforce password strength, complexity, and reuse rules. Configured in /etc/pam.d/ and /etc/security/.
	FAILED AUTHENTICATION ATTEMPTS Limit and monitor failed login attempts. Helps prevent brute-force and password guessing attacks.
	DORMANT ACCOUNTS Accounts that have not been used for a long time increase risk. Identify and disable or remove dormant accounts.
	LOGIN RESTRICTIONS Restrict login access by time, source IP, or TTY. Enhances security for privileged and system accounts.
	REVIEWING PASSWORD STATUS Regularly review password aging, expiration, and account status. Use tools to ensure policies are enforced.
	SECURE SERVICE-ACCOUNT PRACTICES Use strong, long passwords or keys for service accounts. Restrict permissions and avoid interactive logins.

KEY COMMANDS

- `>_ chage -l <user>`
Display password aging information
- `>_ chage -M 90 <user>`
Set password expiration (days)
- `>_ chage -E 2025-12-31 <user>`
Set account expiration date
- `>_ pam-config`
Manage PAM configuration
- `>_ grep "failed password" /var/log/auth.log`
View failed login attempts
- `>_ lastlog`
Check last login time for all users
- `>_ passwd -S <user>`
Show password status
- `>_ usermod -L <user>`
Lock user account
- `>_ usermod -e 1 <user>`
Expire user account immediately

4. FILE AND DIRECTORY PERMISSIONS



OWNER, GROUP, AND OTHER PERMISSIONS

Every file and directory has three types of access: Owner (user), Group, and Other (everyone else). Permissions control what each can do.



READ, WRITE, AND EXECUTE

R (read): View file contents or list directory.
W (write): Modify file or create/delete in directory.
X (execute): Run file or access directory.



NUMERIC AND SYMBOLIC PERMISSIONS

Numeric: Use numbers (0-7) to represent permissions.
Symbolic: Use letters (r, w, x) to set permissions.
Example: 755 = rwxr-xr-x



CHMOD, CHOWN, CHGRP

chmod : Change file or directory permissions
chown : Change file or directory owner
chgrp : Change file or directory group



DIRECTORY PERMISSION BEHAVIOR

Read (r): List files in the directory
Write (w): Create, delete, or rename files
Execute (x): Enter and access the directory
Without x, you cannot access files inside.



UMASK

The umask sets default permissions for new files and directories.
It determines what permissions are removed by default.
Check with: `umask`



FINDING DANGEROUS PERMISSIONS

Look for world-writable files and directories.
Use tools to find risky permissions.
Example: `chmod 777` or writable system files.
Command: `find / -type f -perm -002 -ls`



PRODUCTION PERMISSION MISTAKES

- Using 777 permissions
- Making sensitive files world-readable
- Incorrect ownership of application files
- Writable permissions on critical system directories
- Ignoring umask settings

PERMISSION REFERENCE

PERMISSION	SYMBOL	NUMBER
READ	r	4
WRITE	w	2
EXECUTE	x	1
NONE	-	0

EXAMPLES

FILE: script.sh

`-rw-r--r--` (644)

FILE: application.log

`-rw-rw----` (660)

DIRECTORY: /var/www

`drwxr-xr-x` (755)

DIRECTORY: /data

`drwx-----` (700)

QUICK CHECK COMMANDS

`> ls -l`
List files with permissions

`> namei -l /path/to/file`
Show full path permissions

`> stat file.txt`
Detailed file information

`> find / -type d -perm -002`
Find world-writable directories

`> find / -type f -perm -o+w`
Find world-writable files

5. SUID, SGID, STICKY BIT, AND ACLs

 SUID	SUID EXECUTABLES When a file has the SUID bit set, it runs with the permissions of the file owner (usually root). Risk: Can be abused to gain elevated privileges. Check SUID files regularly.
 SGID	SGID FILES AND DIRECTORIES SGID on files: Runs with the group owner's privileges. SGID on directories: New files inherit the directory's group ownership. Helps in controlled group collaboration.
 STICKY BIT	STICKY BIT On directories, only the file owner, directory owner, or root can delete or rename files. Common on /tmp to prevent users from deleting each other's files.
 ACLs	ACCESS CONTROL LISTS (ACLs) Provide fine-grained permissions beyond the traditional user/group/other model. Useful for granting access to specific users without changing ownership.
 GETFACL AND SETFACL	GETFACL AND SETFACL getfacl : View ACLs on files and directories setfacl : Modify ACLs for fine-grained access ACLs override standard permissions.
 SEARCH	SEARCHING FOR SUID AND SGID BINARIES Regularly search for SUID/SGID files to detect unexpected privilege escalation paths. New or unknown files can be a security risk.
 RISK	UNEXPECTED PRIVILEGE PATHS Attackers exploit misconfigured SUID/SGID files or writable paths to escalate privileges. Always verify owner, permissions, and purpose.
 AUDIT	AUDITING SPECIAL PERMISSIONS Maintain an inventory of all special permission files. Review regularly and document approved exceptions. Remove unnecessary permissions immediately.

SPECIAL PERMISSION TYPES

- S** SUID (4xxx)
Run as file owner
- S** SGID (2xxx)
Run as group owner
- T** STICKY (1xxx)
Restricted deletion
- A** ACLs
Extended permissions

FIND SUID/SGID FILES

Find all SUID files:

```
> find / -type f \( -perm -4000 \) -exec ls -l {} \; 2>/dev/null
```

Find all SGID files:

```
> find / -type f \( -perm -2000 \) -exec ls -l {} \; 2>/dev/null
```

Find SUID and SGID in common paths:

```
> find /bin /sbin /usr/bin /usr/sbin /usr/local/bin -type f \( -perm -4000 -o -perm -2000 \) -ls 2>/dev/null
```

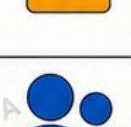
ACL COMMANDS EXAMPLES

```
> getfacl /path/to/file  
# View ACLs  
  
> setfacl -m u:user:rw /path/to/file  
# Add ACL entry  
  
> setfacl -x u:user /path/to/file  
# Remove ACL entry  
  
> setfacl -b /path/to/file  
# Remove all ACLs
```

AUDIT CHECKLIST

- ✓ Review all SUID, SGID, and Sticky files
- ✓ Verify file purpose and ownership
- ✓ Ensure directories with SGID are required
- ✓ Check ACLs for unnecessary access
- ✓ Document approved exceptions
- ✓ Remove or fix unnecessary entries

6. SUDO AND PRIVILEGE ESCALATION SECURITY

	ROOT ACCESS RISKS Root has unlimited power on the system. A single mistake or compromise can lead to full system takeover. Limit and monitor root access carefully.
	/ETC/SUDOERS Main sudo configuration file. Controls which users or groups can run which commands as root or another user. Always edit with visudo, never a text editor.
	/ETC/SUDOERS.D/ Directory for additional sudo configuration files. Used to add custom rules without modifying the main sudoers file. Each file in this directory is included automatically.
	VISUDO Safe way to edit sudoers files. Syntax is checked before saving. Prevents locking yourself out of sudo access. Always use: visudo or visudo -f <file>
	COMMAND-SPECIFIC SUDO ACCESS Grant users access to only the commands they need. Example: Allow restart of specific services without full root access. Reduces the attack surface.
	DANGEROUS SUDO CONFIGURATIONS Avoid broad permissions and wildcard commands. Dangerous examples: • ALL=(ALL) ALL • ALL=(ALL) NOPASSWD: ALL • Command wildcards like /bin/* These can lead to full privilege escalation.
	NOPASSWD RISKS NOPASSWD allows running commands without entering a password. If misconfigured, attackers can escalate privileges without any authentication. Use only when absolutely necessary.
	REVIEWING SUDO PRIVILEGES Regularly review who has sudo access and what they can run. Remove unused or excessive privileges. Use audit logs to monitor sudo activity.
	LEAST-PRIVILEGE ADMINISTRATION Grant only the minimum access required. Remove elevated access when not needed. Use groups, command restrictions, and logging to enforce least-privilege principles.

SUDO COMMAND EXAMPLES

-  **sudo -l**
List allowed sudo commands for current user
-  **sudo -ll**
List all privileges in detail
-  **sudo -u <user> <command>**
Run command as another user
-  **visudo**
Edit /etc/sudoers safely
-  **visudo -f /etc/sudoers.d/<file>**
Edit a file in sudoers.d directory
-  **sudo -k**
Invalidate password timestamp (next sudo will ask password)
-  **sudo -v**
Refresh the sudo timestamp (extend login session)

SUDO SECURITY CHECKLIST

- ✓ Limit users with sudo access
- ✓ Use command-specific permissions
- ✓ Avoid NOPASSWD unless required
- ✓ Do not use wildcards carelessly
- ✓ Review /etc/sudoers regularly
- ✓ Audit sudo usage in logs
- ✓ Remove unnecessary privileges
- ✓ Use least-privilege principles
- ✓ Test configurations before deployment

VERIQTA

7. SSH SERVER HARDENING



SSH ATTACK SURFACE

SSH is a common target for attackers. Reduce exposure by limiting access, strengthening authentication, and monitoring activity.



KEY-BASED AUTHENTICATION

Use SSH keys instead of passwords. Stronger, more secure, and resistant to brute-force attacks.



DISABLING ROOT SSH LOGIN

Do not allow root to login via SSH. Use a regular user and sudo for administrative tasks.



DISABLING PASSWORD AUTHENTICATION

Disable password-based login. Prevents brute-force and credential stuffing attacks.



SSHD_CONFIG

Main SSH server configuration file. Harden settings in /etc/ssh/sshd_config and reload SSH service safely.



RESTRICTING USERS AND GROUPS

Allow only specific users or groups to access SSH. Deny all others.



SSH KEY PERMISSIONS

Private key (id_rsa): 600
Public key (id_rsa.pub): 644
~/.ssh directory: 700
authorized_keys file: 600
Wrong permissions = login failure.



IDLE-SESSION CONTROLS

Set timeouts for idle sessions. Disconnect inactive sessions automatically to reduce risk.



TESTING CONFIGURATION BEFORE RESTART

Always validate SSH configuration before restarting to avoid lockout. Use test mode: `sshd -t`



REVIEWING SSH AUTHENTICATION LOGS

Monitor SSH login attempts, failed logins, and suspicious activity regularly. Logs help detect brute-force attacks early.

IMPORTANT SSHD_CONFIG SETTINGS

Port	22
Protocol	2
PermitRootLogin	no
PasswordAuthentication	no
ChallengeResponseAuthentication	no
UsePAM	yes
AllowUsers	user1 user2
AllowGroups	sshusers
X11Forwarding	no
MaxAuthTries	3
LoginGraceTime	30s
ClientAliveInterval	300
ClientAliveCountMax	2

SSH KEY GENERATION (ON CLIENT)

```
ssh-keygen -t ed25519 -C "your_email@domain.com" -f ~/.ssh/id_ed25519
```

COPY PUBLIC KEY TO SERVER

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@server_ip
```

OR

```
cat ~/.ssh/id_ed25519.pub | ssh user@server_ip \"mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys\"
```

VALIDATE SSH CONFIGURATION

```
sshd -t  
# If no output, configuration is valid  
# If errors, fix them before restarting
```

RELOAD SSH SERVICE

```
sudo systemctl reload sshd  
# Or restart if necessary  
sudo systemctl restart sshd
```

REVIEW AUTHENTICATION LOGS

```
sudo journalctl -u ssh --since "1 hour ago"  
sudo grep "sshd" /var/log/auth.log  
sudo grep "Failed password" /var/log/auth.log
```

8. LINUX SERVICES AND SYSTEMD SECURITY

	IDENTIFYING UNNECESSARY SERVICES Many services start automatically by default. Remove or disable services you do not need. Every unnecessary service increases your attack surface.
 systemctl	SYSTEMCTL The primary tool for managing systemd services. Used to start, stop, enable, disable, and inspect services.
 ENABLED  RUNNING	ENABLED VERSUS RUNNING SERVICES Enabled: Starts automatically at boot. Running: Currently active in memory. A service can be enabled, running, both, or neither.
	DISABLING UNUSED SERVICES Disable services to prevent them from starting at boot. Stop services that are not needed now. Example: <code>systemctl disable --now <service></code>
	SERVICE USERS Services should run as an unprivileged user. Avoid running services as root. Check the "User=" and "Group=" settings in unit files.
	SYSTEMD UNIT PERMISSIONS Unit files control how services run. Protect unit files from unauthorized changes. Typical path: <code>/etc/systemd/system/</code> . Use proper file permissions.
	SERVICE ISOLATION Use systemd sandboxing to limit damage. Options: ProtectSystem, ProtectHome, PrivateTmp, NoNewPrivileges, RestrictAddressFamilies, and more. Stronger isolation = better security.
	RESTART POLICIES Configure how services behave when they fail. Common policies: no, on-failure, on-abnormal, on-watchdog, always. Use <code>RestartSec=</code> to set delay between restarts.
	INSPECTING FAILED OR SUSPICIOUS SERVICES Check failed services and view recent logs. Look for crash loops, repeated failures, or unexpected behavior. Use: <code>systemctl status <service></code> and <code>journalctl -u <service></code>
	REDUCING SERVICE ATTACK SURFACE Run only what you need. Disable, mask, or remove unneeded services. Keep services updated and properly configured. Minimize privileges and isolate workloads. Fewer services = fewer risks.

COMMON SYSTEMCTL COMMANDS

- > `systemctl list-units --type=service`
List all services
- > `systemctl list-unit-files --type=service`
Show enabled/disabled services
- > `systemctl status <service>`
Show service status and logs
- > `systemctl start <service>`
Start a service
- > `systemctl stop <service>`
Stop a service
- > `systemctl restart <service>`
Restart a service
- > `systemctl enable <service>`
Enable at boot
- > `systemctl disable <service>`
Disable from boot
- > `systemctl mask <service>`
Prevent service from starting
- > `systemctl unmask <service>`
Unmask a service

CHECK ENABLED VS RUNNING SERVICES

List enabled services:

```
systemctl list-unit-files --type=service --state=enabled
```

List running services:

```
systemctl list-units --type=service --state=running
```

Compare both to find unused services.

VIEW SERVICE DETAILS

```
systemctl cat <service>
```

Show full unit file

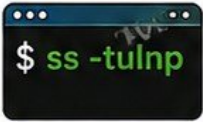
```
systemctl show <service>
```

Show all properties

BEST PRACTICES

- ✓ Disable services you do not use.
- ✓ Run services as non-root users.
- ✓ Harden services with isolation options.
- ✓ Limit network and filesystem access.
- ✓ Monitor service logs and failures.
- ✓ Keep services updated.
- ✓ Review services regularly.
- ✓ Follow least-privilege principles.

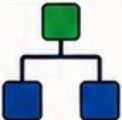
9. NETWORK EXPOSURE AND OPEN PORTS

	LISTENING SOCKETS Listening sockets are endpoints that accept incoming connections or packets on the system.
	TCP AND UDP EXPOSURE Open TCP ports establish connections. Open UDP ports receive datagrams. Both increase the attack surface.
	SS ss is the modern tool to view socket statistics, listening ports, and network connections. Use it to identify what is listening on the system.
	Lsof Lsof lists open files and the processes that opened them, including network sockets. Useful for identifying listening ports and connections.
	IDENTIFYING PROCESSES BEHIND PORTS Find which process is listening on a specific port. Understand the service, user, and risk associated with that process.
	LOOPBACK VERSUS PUBLIC INTERFACES Services bound to 127.0.0.1 are only accessible locally. Services bound to 0.0.0.0 or public IPs are accessible from the network or internet.
	UNEXPECTED LISTENING SERVICES Unexpected or unnecessary services listening on ports may indicate misconfiguration or potential compromise. Always review and minimize exposed services.
	NETWORK EXPOSURE INVESTIGATION Investigate open ports, services, and connections. Correlate with logs and configurations to assess the security impact.
	PRODUCTION PORT AUDITING Regularly audit open ports and listening services. Validate that only required services are exposed. Document, monitor, and review continuously.

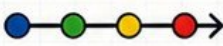
10. LINUX FIREWALL SECURITY

	HOST-BASED FIREWALL PURPOSE A host-based firewall controls network traffic to and from your Linux system. It blocks unauthorized access, reduces attack surface, and enforces security policies at the host level.
	NFTABLES nftables is the modern Linux packet filtering framework. It is powerful, fast, and replaces legacy iptables. It supports complex rules, sets, maps, and stateful filtering.
	IPTABLES CONCEPTS iptables is the legacy firewall tool based on tables, chains, and rules. Key tables: filter, nat, mangle, raw. Key chains: INPUT, OUTPUT, FORWARD.
	UFW AND FIREWALLD UFW is a simple frontend for iptables/nftables. firewalld provides a dynamic firewall with zones, services, and runtime management.
	DEFAULT-DENY STRATEGY Start with default deny for incoming connections. Allow only what is required. This minimizes risk by blocking everything except explicitly allowed traffic.
	INBOUND VERSUS OUTBOUND RULES Inbound rules control traffic coming into the system. Outbound rules control traffic leaving the system. Both should be defined intentionally.
	RESTRICTING ADMINISTRATIVE PORTS Limit access to administrative ports (SSH, RDP, etc.) to trusted IPs or networks only. Never expose admin ports to the public internet.
	RULE ORDERING Firewall rules are processed in order from top to bottom. Place specific allow rules before general deny rules. Incorrect ordering can create security gaps.
	PERSISTENT FIREWALL CONFIGURATION Firewall rules must survive reboots. Use appropriate tools to save rules: nft list ruleset > file, iptables-save, ufw enable, firewalld --permanent.
	TESTING RULES WITHOUT LOCKING YOURSELF OUT Always test firewall changes carefully. Use a second SSH session or console access. Apply changes gradually and verify before closing your session.

11. PROCESS AND RUNTIME SECURITY

	PROCESS OWNERSHIP Every process runs as a specific user and group. Review process ownership to ensure least privilege and detect suspicious activity.
	PARENT AND CHILD PROCESSES Processes are created by other processes. Understanding parent-child relationships helps trace the origin of a process.
	PRIVILEGED PROCESSES Processes running as root or with elevated capabilities pose higher risk. Review and restrict privileged processes.
	PS, TOP, PSTREE Use ps to view processes, top for live monitoring, and pstree to visualize process trees. Essential tools for runtime visibility.
	/PROC The /proc filesystem provides real-time information about processes, memory, mounts, network, and more. Powerful for investigation.
	SUSPICIOUS COMMAND LINES Review process command lines. Look for obfuscated, encoded, or unusual commands that may indicate malicious activity.
	UNEXPECTED BINARIES Processes running from unusual locations (e.g., /tmp, /dev/shm) or unknown binaries may indicate compromise.
	LONG-RUNNING PROCESSES Processes running longer than expected can indicate stuck services or persistence mechanisms. Review and validate regularly.
	PROCESSES LISTENING ON THE NETWORK Identify processes that have open sockets. Ensure only required services are listening on expected ports and interfaces.
	INVESTIGATING ABNORMAL RUNTIME ACTIVITY Correlate process behavior, network activity, and logs to detect anomalies. Investigate immediately and contain threats.

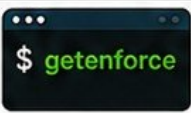
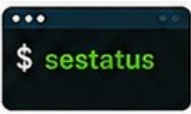
12. LOGGING AND SECURITY INVESTIGATION

	LINUX AUTHENTICATION LOGS Authentication logs record user logins, logouts, and security-relevant events. Key sources: <code>/var/log/auth.log</code> , <code>/var/log/secure</code> .
	JOURNALCTL <code>journalctl</code> is the systemd log manager. View system, service, kernel, and boot logs. Powerful filtering with time, unit, priority, and more.
	/VAR/LOG Traditional log files stored in <code>/var/log</code> . Contains authentication, system, application, and service logs.
	SSH LOGIN EVENTS Track successful and failed SSH logins. Logs show IP address, user, time, and authentication method. Helps detect brute-force and suspicious access.
	SUDO ACTIVITY Monitor <code>sudo</code> command usage and privilege escalation. Logs show who ran what command and when. Essential for auditing privileged access.
	FAILED AUTHENTICATION Failed login attempts indicate attacks or misconfiguration. Track repeated failures from the same IP or user. Helps identify brute-force and password attacks.
	SERVICE FAILURES Service errors can impact availability and security. Logs show crashes, dependency issues, and start failures. Helps detect misconfigurations and potential attacks.
	KERNEL MESSAGES Kernel logs report hardware, driver, and system events. Important for detecting panics, OOM kills, and security issues. Access via <code>dmesg</code> or <code>journalctl -k</code> .
	SEARCHING AND FILTERING LOGS Use tools like <code>grep</code> , <code>awk</code> , <code>sed</code> , and <code>journalctl</code> filters. Filter by time, user, IP, service, priority, or keywords. Effective searching speeds up investigation.
	BUILDING AN INCIDENT TIMELINE Correlate events from multiple logs to understand what happened. Establish sequence: initial access, actions, and impact. A clear timeline is critical for response and reporting.

13. LINUX AUDITING WITH AUDITD

	LINUX AUDIT FRAMEWORK The Linux Audit Framework provides a comprehensive system for monitoring, recording, and reviewing security-relevant events. It operates at the kernel level for high integrity and reliability.
	AUDITD auditd is the user-space service that collects audit events from the kernel and writes them to the audit log. It is highly configurable and essential for security monitoring.
	AUDIT RULES Audit rules define what events to monitor. Rules can be file watches, syscalls, users, commands, or system calls. Use auditctl to load and manage rules.
	WATCHING SENSITIVE FILES Monitor critical files and directories for reads, writes, attribute changes, and deletions. Example: /etc/passwd, /etc/shadow, /etc/sudoers.
	MONITORING PRIVILEGED COMMANDS Track the use of privileged commands such as sudo, su, useradd, chmod, chown, setuid, and more. Helps detect privilege escalation and misuse.
	AUSEARCH ausearch is used to search the audit log. Filter by time, user, event type, file, PID, or command. Essential for incident investigations.
	AUREPORT aureport generates summarized reports from audit logs. Use it for quick insights on logins, commands, files, and SELinux events.
	TRACKING CONFIGURATION CHANGES Monitor changes to critical configuration files. Detect unauthorized modifications to system settings, services, and security policies.
	AUDIT-LOG PROTECTION Protect audit logs from tampering. Set proper permissions, enable log rotation, and forward logs to a secure, centralized location.
	PRACTICAL INVESTIGATION EXAMPLES Real-world scenarios: detecting unauthorized access, tracking file modifications, identifying privilege escalation, and investigating suspicious activity step-by-step.

14. SELINUX AND APPARMOR

 SELinux	MANDATORY ACCESS CONTROL SELinux and AppArmor are Mandatory Access Control (MAC) systems that restrict what processes can do, even if they are compromised or running as root.
 MODES	SELINUX MODES SELinux operates in three main modes that determine how policies are enforced.
	ENFORCING, PERMISSIVE, AND DISABLED • Enforcing: Policies are active and violations are blocked. • Permissive: Policies are active but violations are only logged. • Disabled: SELinux is turned off (not recommended).
 CTX	SELINUX CONTEXTS SELinux uses security contexts (labels) to define what users, roles, types, and levels a process or file has. Contexts control access decisions.
	POLICY VIOLATIONS When a process is blocked by SELinux, an AVC (Access Vector Cache) denial is logged. These logs show what was blocked and why.
 \$ <code>getenforce</code>	GETENFORCE <code>getenforce</code> shows the current SELinux mode. Command: <code>getenforce</code> Example output: Enforcing Permissive Disabled
 \$ <code>sestatus</code>	SESTATUS <code>sestatus</code> displays SELinux status, mode, policy type, and other important details. Command: <code>sestatus</code>
 AppArmor	APPARMOR PROFILES AppArmor uses profiles to restrict programs. Each profile defines what the program can access. Profiles are stored in <code>/etc/apparmor.d/</code>
	ENFORCE VERSUS COMPLAIN MODE • Enforce: Violations are blocked. • Complain: Violations are logged but not blocked. Use complain mode to test profiles before enforcing.
	TROUBLESHOOTING BLOCKED APPLICATIONS SAFELY • Review AVC logs: <code>ausearch -m avc -ts recent</code> • Identify the blocked action and target. • Adjust the policy or profile. • Use permissive or complain mode during troubleshooting. • Return to Enforcing/Enforce mode after validation.

15. PACKAGE, REPOSITORY, AND PATCH SECURITY

	TRUSTED PACKAGE REPOSITORIES • Use official and trusted repositories from your distribution. • Avoid third-party repositories unless absolutely necessary. • Untrusted repositories can introduce malicious packages.
	PACKAGE SIGNATURES • Packages are cryptographically signed by the repository. • The system verifies signatures before installing packages. • Ensures integrity and authenticity of packages.
	SECURITY UPDATES • Security updates fix vulnerabilities and reduce risk. • Apply updates promptly to protect against known threats. • Subscribe to security advisories for your distribution.
	PATCH MANAGEMENT • Establish a process for testing and applying patches. • Use staging environments before production rollout. • Document changes and maintain rollback plans.
	VULNERABILITY EXPOSURE • Outdated packages are a common attack vector. • Vulnerabilities can lead to privilege escalation or system compromise. • Regular patching reduces your attack surface.
	DETECTING OUTDATED PACKAGES • Use tools to list outdated or vulnerable packages. • Examples: <code>dnf check-update</code>, <code>apt list --upgradable</code> • Regularly review and prioritize critical updates.
	AUTOMATIC SECURITY UPDATES • Enable automatic updates for critical security patches. • Tools: <code>unattended-upgrades</code>, <code>dnf-automatic</code>, <code>yum-cron</code> • Balance automation with change control policies.
	REPOSITORY CONFIGURATION • Configure repositories securely and verify URLs. • Disable unnecessary or unused repositories. • Ensure GPG keys are imported and trusted.
	REMOVING UNNECESSARY PACKAGES • Remove unused packages and dependencies. • Reduce system bloat and minimize attack surface. • Tools: <code>dnf remove</code>, <code>apt autoremove</code>
	PRODUCTION PATCHING STRATEGY • Define patch windows and maintenance schedules. • Prioritize critical and high-risk vulnerabilities. • Test, deploy, verify, and monitor after patching. • Maintain documentation and audit patch compliance.

PAGE 16: FILE INTEGRITY AND PERSISTENCE DETECTION

KEY TOPICS



CRITICAL SYSTEM FILES



UNEXPECTED FILE
MODIFICATIONS



FILE HASHES



SHA256SUM



FILE INTEGRITY
MONITORING



CRON PERSISTENCE



SYSTEMD PERSISTENCE



SHELL STARTUP FILES



SUSPICIOUS BINARIES



DETECTING UNAUTHORIZED
CHANGES

CRITICAL SYSTEM FILES TO MONITOR

- 📁 /etc/passwd
- 📁 /etc/shadow
- 📁 /etc/sudoers
- 📁 /etc/ssh/ssh_config
- 📁 /bin, /sbin, /usr/bin, /usr/sbin
- 📁 /etc/systemd/system/
- 📁 /boot/
- 📁 /var/spool/cron/
- 📁 /root/.bashrc, /root/.ssh/



FILE HASHES WITH SHA256SUM

Generate SHA256 hash for a file
sha256sum /etc/passwd

Generate SHA256 hash for a directory (recursive)
sha256sum -r /etc > etc.sha256

Verify against known good hash list
sha256sum -c etc.sha256



FILE INTEGRITY MONITORING

- Regularly hash critical files.
- Store hashes securely.
- Compare hashes to detect changes.
- Use tools like AIDE, Tripwire, or OSSEC.



PERSISTENCE LOCATIONS



CRON PERSISTENCE
Check for malicious or
unexpected cron jobs.

```
crontab -l
ls -la /etc/cron.*
ls -la /var/spool/cron/
```



SYSTEMD PERSISTENCE
Malicious or unknown
systemd services.

```
systemctl list-unit-files --type=service
ls -la /etc/systemd/system/
```



SHELL STARTUP FILES
Check for suspicious
commands in startup
scripts.

```
cat ~/.bashrc
cat ~/.bash_profile
ls -la /etc/profile.d/
```

DETECTING UNAUTHORIZED CHANGES



- Monitor critical files and directories.
- Track new or modified binaries.
- Review file ownership and permissions.
- Alert on unexpected changes.
- Investigate immediately.



PROTECT. MONITOR. VERIFY.
INTEGRITY TODAY, SECURITY FOREVER.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

PAGE 17: SECRETS, CREDENTIALS, AND SENSITIVE DATA



1. SECRETS STORED IN CONFIGURATION FILES



- Avoid storing passwords, keys, and tokens in config files.
- Common files:
/etc/*.conf
~/.config/*
/var/www/html/*.php
.env, settings.json

2. ENVIRONMENT VARIABLES



- Environment variables may contain secrets.
- Check:
env
printenv
cat /proc/*/environ

3. SSH PRIVATE KEYS



- Protect private keys at all times.
- Default key location:
~/.ssh/id_rsa
~/.ssh/
- Key permissions should be 600.
- Never share private keys.

4. API KEYS AND TOKENS



- API keys grant access to services.
- Store securely.
- Avoid hardcoding.
- Rotate regularly.

5. DATABASE CREDENTIALS



- Credentials should not be stored in plain text.
- Check config files, scripts, and environment variables.
- Use dedicated DB users with least privilege.

6. FILE PERMISSION MISTAKES



- Incorrect permissions can expose secrets.
- Sensitive files should be readable by owner only.
- Check file permissions:
ls -l <file>
- Fix with:
chmod 600 <file>

7. SHELL HISTORY EXPOSURE



- Commands containing secrets may be saved in history.
- Check history files:
~/.bash_history
/root/.bash_history
- Clear history when needed:
history -c

8. PROCESS ARGUMENT EXPOSURE



- Secrets passed as arguments can be visible to other users.
- Check running processes:
ps -ef
- Avoid passing secrets via command line.

9. SECRET ROTATION



- Rotate credentials regularly.
- Revoke old keys and tokens.
- Automate rotation where possible.
- Document rotation procedures.
- Monitor for unused credentials.

10. SEARCHING SYSTEMS FOR EXPOSED CREDENTIALS

Search for common patterns:

```
grep -rni "password" /
grep -rni "passwd" /
grep -rni "api_key" /
grep -rni "secret" /
grep -rni "token" /
```

Use tools:

- grep
- ripgrep (rg)
- truffleHog
- git-secrets
- gitleaks



**EXPOSED SECRETS CAN LEAD TO COMPLETE SYSTEM COMPROMISE.
STORE SECURELY. LIMIT ACCESS. ROTATE REGULARLY.**



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

PAGE 18: LINUX SECURITY HARDENING BASELINE



1



SECURE INSTALLATION BASELINE

- Install a minimal OS.
- Use trusted repositories only.
- Configure hostname, timezone, and NTP.

2



REMOVE UNNECESSARY SOFTWARE

- Remove unused packages.
- Avoid installing unnecessary tools.
- Reduce attack surface.

3



DISABLE UNUSED SERVICES

- Identify unnecessary services.
- Disable and stop them.
- Prevent unwanted exposure.

4



RESTRICT PRIVILEGED ACCESS

- Limit root access.
- Use sudo with least privilege.
- Review and tighten sudoers rules.

5



HARDEN SSH

- Disable root login.
- Disable password authentication.
- Use key-based authentication.
- Restrict users and groups.

6



SECURE FILESYSTEM PERMISSIONS

- Set correct ownership and permissions.
- Use least-privilege file access.
- Audit critical directories and files.

7



CONFIGURE FIREWALL RULES

- Apply default deny policy.
- Allow only required services.
- Restrict admin ports.
- Review rules regularly.

8



ENABLE AUDITING

- Install and configure auditd.
- Monitor privileged commands.
- Watch critical files and directories.
- Review audit logs regularly.

9



APPLY SECURITY UPDATES

- Keep system and packages updated.
- Enable automatic security updates where possible.
- Regularly check for vulnerabilities.

10



CIS BENCHMARK CONCEPTS

- Follow CIS Benchmark guidelines.
- Apply secure configurations.
- Validate settings against CIS controls.
- Continuously improve security posture.

11



DOCUMENTING APPROVED EXCEPTIONS

- Document any security exceptions.
- Include business justification.
- Review and revalidate regularly.



A STRONG BASELINE = A SECURE LINUX SYSTEM

Hardening is not a one-time task.

Continuously review, validate, and improve your security posture.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

PAGE 19:

PRODUCTION SECURITY INCIDENT INVESTIGATION



SCENARIO:
A LINUX SERVER SHOWS SUSPICIOUS ACTIVITY.

INVESTIGATION STEPS



1 IDENTIFY ACTIVE USERS

```
who; whoami; w
```



2 REVIEW RECENT LOGINS

```
last; last -a; lastlog
```



3 CHECK FAILED SSH ATTEMPTS

```
grep "Failed password" /var/log/auth.log
```



4 INSPECT SUDO ACTIVITY

```
sudo journalctl -u sudo;  
ausearch -m USER_CMD
```



5 REVIEW RUNNING PROCESSES

```
ps aux; top; pstree -p
```



6 IDENTIFY OPEN PORTS

```
ss -tulpen; netstat -tulpen
```



7 INSPECT NETWORK CONNECTIONS

```
ss -antp; lsof -i -P -n
```



8 SEARCH AUTHENTICATION LOGS

```
grep -E "sshd|login|su|sudo" /var/log/auth.log
```



9 CHECK CRON JOBS AND SYSTEMD UNITS

```
crontab -l; ls -la /etc/cron.*  
systemctl list-unit-files  
--type=service
```



10 FIND RECENTLY MODIFIED FILES

```
find / -type f -mtime -1 -ls  
(Adjust time as needed)
```



11 LOOK FOR SUID BINARIES

```
find / -type f -perm -4000  
-ls 2>/dev/null
```



12 CHECK NEW OR MODIFIED USER ACCOUNTS

```
cut -d: -f1 /etc/passwd  
getent passwd
```



13 PRESERVE EVIDENCE BEFORE REMEDIATION

```
cp /var/log/auth.log /root/  
evidence_auth_$(date +%F).log  
dmesg > /root/evidence_dmesg_$(date +%F).log
```



**ALWAYS PRESERVE EVIDENCE BEFORE TAKING CORRECTIVE ACTION.
DOCUMENT EVERYTHING.**

PAGE 20: LINUX SECURITY PRODUCTION CHECKLIST



FINAL VERIFICATION

- | | |
|--|--|
| 1 USER ACCOUNTS REVIEWED ☐ | 11 AUDITD CONFIGURED ☐ |
| 2 PRIVILEGED ACCESS RESTRICTED ☐ | 12 SELINUX OR APPARMOR ENFORCED WHERE APPROPRIATE ☐ |
| 3 PASSWORD POLICY CONFIGURED ☐ | 13 SECURITY PATCHES CURRENT ☐ |
| 4 SSH HARDENED ☐ | 14 SECRETS PROTECTED ☐ |
| 5 FILE PERMISSIONS AUDITED ☐ | 15 PERSISTENCE MECHANISMS REVIEWED ☐ |
| 6 SUID AND SGID FILES REVIEWED ☐ | 16 FILE INTEGRITY MONITORED ☐ |
| 7 UNNECESSARY SERVICES DISABLED ☐ | 17 BACKUPS PROTECTED AND TESTED ☐ |
| 8 OPEN PORTS VERIFIED ☐ | 18 TIME SYNCHRONIZATION CONFIGURED ☐ |
| 9 FIREWALL ENABLED ☐ | 19 SECURITY ALERTS ESTABLISHED ☐ |
| 10 LOGS CENTRALIZED OR RETAINED ☐ | 20 INCIDENT RESPONSE PROCEDURE DOCUMENTED ☐ |



**A SECURE LINUX SYSTEM IS BUILT BY DESIGN,
MAINTAINED BY DISCIPLINE,
AND PROTECTED BY CONTINUOUS VERIFICATION.**



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta